

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import IsolationForest
from sklearn.metrics import pairwise_distances

import joblib

print("Libraries loaded successfully!")
```

Libraries loaded successfully!

```
# Paths to datasets
BREAKDOWN_PATH = "/content/Breakdown.csv"
LEAKAGE_PATH = "/content/LeakageIV.csv"
TURNON_PATH = "/content/TurnOn.csv"

# Load datasets
breakdown = pd.read_csv(BREAKDOWN_PATH)
leakage = pd.read_csv(LEAKAGE_PATH)
turnon = pd.read_csv(TURNON_PATH)

print("Breakdown:")
print(breakdown.head())

print("\nLeakage IV:")
print(leakage.head())

print("\nTurn-On:")
print(turnon.head())
```

```
Breakdown:
  Collector_Emitter_Voltage_Vce  Leakage_Current_Ic
0                2.020181          5.885971e-09
1                2.040083          5.927931e-09
2                2.060686          5.913162e-09
3                2.080598          5.896354e-09
4                2.100827          6.043535e-09
```

```
Leakage IV:
  Applied_Voltage  Leakage_Current
0        1.007554      3.850000e-09
1        4.011488      1.030000e-08
2        7.021188      1.870000e-08
3       10.063710      2.690000e-08
4       13.051750      3.490000e-08
```

```
Turn-On:
  Gate_Voltage  Collector_Current
0        2.020181          5.885971e-09
1        2.040083          5.927931e-09
2        2.060686          5.913162e-09
3        2.080598          5.896354e-09
4        2.100827          6.043535e-09
```

```
print("BREAKDOWN SHAPE:", breakdown.shape)
print("LEAKAGE SHAPE:", leakage.shape)
print("TURN-ON SHAPE:", turnon.shape)
```

```
print("\nBreakdown columns:")
print(breakdown.columns.tolist())
```

```
print("\nLeakage columns:")
print(leakage.columns.tolist())
```

```
print("\nTurn-On columns:")
print(turnon.columns.tolist())
```

```
BREAKDOWN SHAPE: (3790, 2)
LEAKAGE SHAPE: (3790, 2)
TURN-ON SHAPE: (3790, 2)
```

```
Breakdown columns:
['Collector_Emitter_Voltage_Vce', 'Leakage_Current_Ic']
```

```
Leakage columns:
['Applied_Voltage', 'Leakage_Current']
```

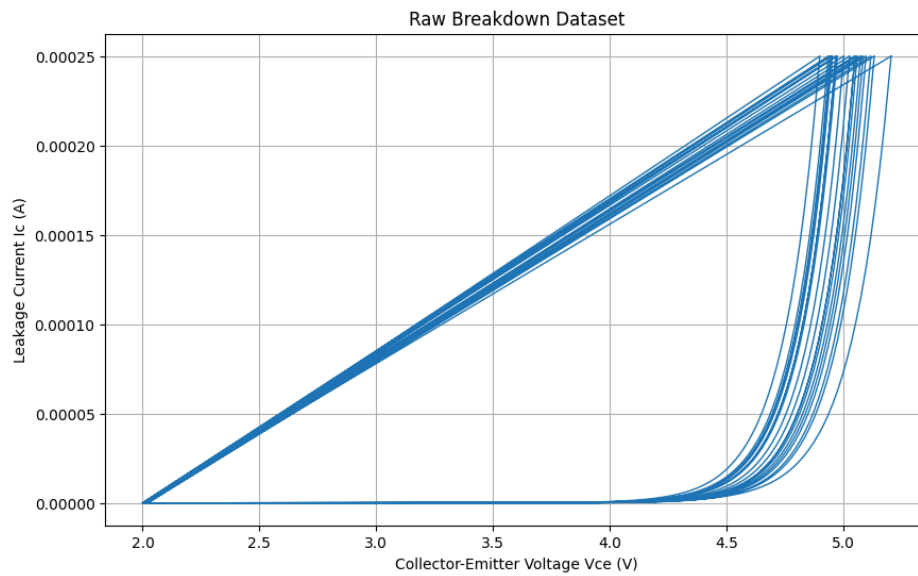
```
Turn-On columns:
['Gate_Voltage', 'Collector_Current']
```

```
plt.figure(figsize=(10, 6))

plt.plot(
    breakdown["Collector_Emitter_Voltage_Vce"],
    breakdown["Leakage_Current_Ic"],
    linewidth=1
)

plt.xlabel("Collector-Emitter Voltage Vce (V)")
plt.ylabel("Leakage Current Ic (A)")
plt.title("Raw Breakdown Dataset")
plt.grid(True)

plt.show()
```

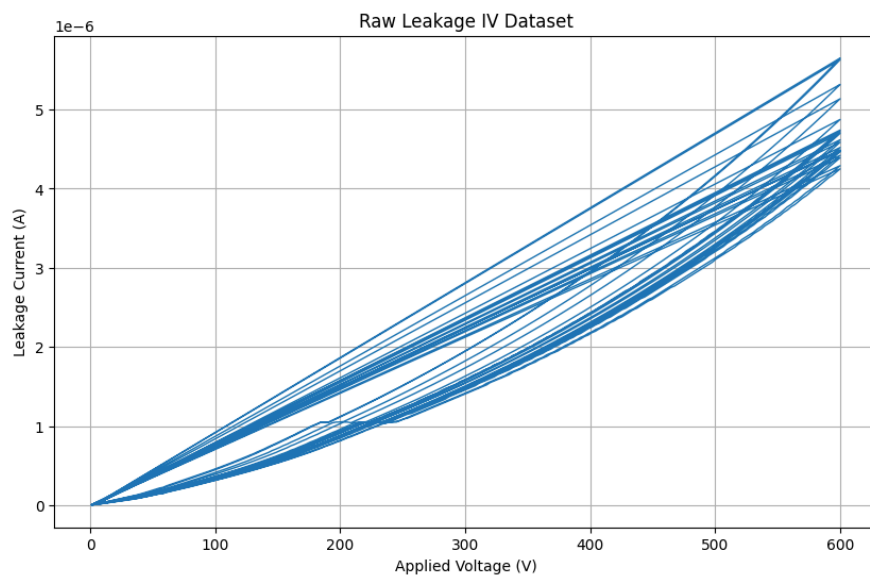


```
plt.figure(figsize=(10, 6))

plt.plot(
    leakage["Applied_Voltage"],
    leakage["Leakage_Current"],
    linewidth=1
)

plt.xlabel("Applied Voltage (V)")
plt.ylabel("Leakage Current (A)")
plt.title("Raw Leakage IV Dataset")
plt.grid(True)

plt.show()
```

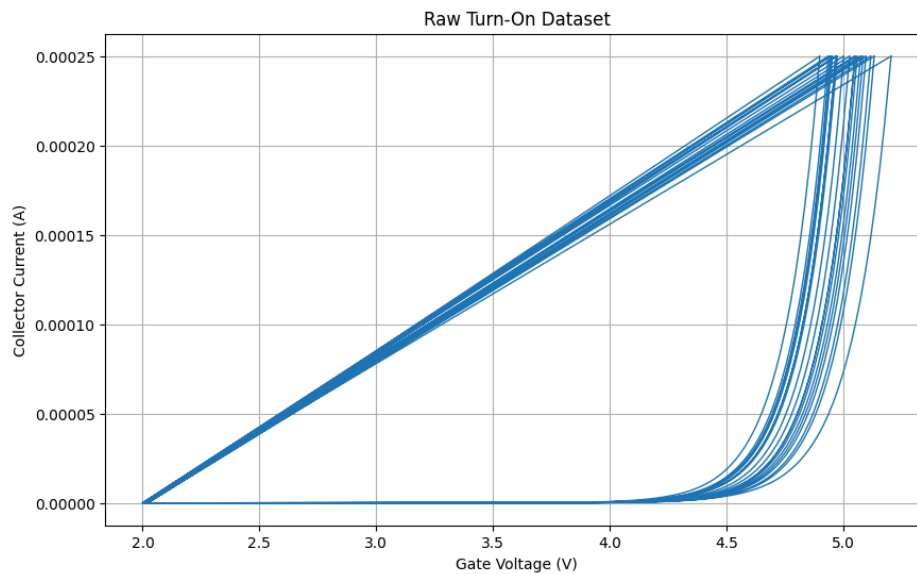


```
plt.figure(figsize=(10, 6))

plt.plot(
    turnon["Gate_Voltage"],
    turnon["Collector_Current"],
    linewidth=1
)

plt.xlabel("Gate Voltage (V)")
plt.ylabel("Collector Current (A)")
plt.title("Raw Turn-On Dataset")
plt.grid(True)

plt.show()
```



```
def detect_curves(df, voltage_column, reset_threshold=0.5):
    df = df.copy()
    voltage = df[voltage_column].values
    curve_id = np.zeros(len(df), dtype=int)
    current_curve = 0
    for i in range(1, len(voltage)):
        # New curve only when voltage makes a LARGE reset
        if voltage[i] < voltage[i - 1] - reset_threshold:
            current_curve += 1
        curve_id[i] = current_curve
    df["curve_id"] = curve_id
    return df
```

```
breakdown_curves = detect_curves(
    breakdown,
    "Collector_Emitter_Voltage_Vce"
)

leakage_curves = detect_curves(
    leakage,
    "Applied_Voltage"
)

turnon_curves = detect_curves(
    turnon,
    "Gate_Voltage"
)

print("Curve detection completed!")
```

Curve detection completed!

```
print(
    "Breakdown curves:",
    breakdown_curves["curve_id"].nunique()
)

print(
    "Leakage IV curves:",
    leakage_curves["curve_id"].nunique()
)

print(
    "Turn-On curves:",
    turnon_curves["curve_id"].nunique()
)
```

Breakdown curves: 19
Leakage IV curves: 19
Turn-On curves: 19

✎ FEATURE EXTRACTION

```
import numpy as np
import pandas as pd

def extract_curve_features(
    df,
    voltage_col,
```

```

current_col
):

features = []

for curve_id, curve in df.groupby("curve_id"):

    # Sort by voltage
    curve = curve.sort_values(voltage_col)

    voltage = curve[voltage_col].values
    current = curve[current_col].values

    # Basic statistics
    min_voltage = np.min(voltage)
    max_voltage = np.max(voltage)

    min_current = np.min(current)
    max_current = np.max(current)

    mean_current = np.mean(current)
    std_current = np.std(current)

    # Numerical derivative dI/dV
    slopes = np.gradient(current, voltage)

    max_slope = np.max(slopes)
    mean_slope = np.mean(slopes)

    # Voltage corresponding to maximum slope
    knee_index = np.argmax(slopes)
    knee_voltage = voltage[knee_index]

    # Area under I-V curve
    area = np.trapezoid(current, voltage)

    features.append({
        "curve_id": curve_id,

        "min_voltage": min_voltage,
        "max_voltage": max_voltage,

        "min_current": min_current,
        "max_current": max_current,

        "mean_current": mean_current,
        "std_current": std_current,

        "max_slope": max_slope,
        "mean_slope": mean_slope,

        "knee_voltage": knee_voltage,

        "curve_area": area
    })

return pd.DataFrame(features)

```

```

breakdown_features = extract_curve_features(
    breakdown_curves,
    "Collector_Emitter_Voltage_Vce",
    "Leakage_Current_Ic"
)

print(breakdown_features)

```

	curve_id	min_voltage	max_voltage	min_current	max_current	\
0	0	2.020181	4.974007	5.885971e-09	0.00025	
1	1	2.020185	5.026770	5.937019e-09	0.00025	
2	2	2.020205	5.049754	6.093055e-09	0.00025	
3	3	2.020159	5.084847	6.105468e-09	0.00025	
4	4	2.020186	5.097120	6.623396e-09	0.00025	
5	5	2.020182	5.132773	7.062614e-09	0.00025	
6	6	2.020198	5.074069	6.591616e-09	0.00025	
7	7	2.020194	5.117679	6.587623e-09	0.00025	
8	8	2.020174	4.899527	6.104020e-09	0.00025	
9	9	2.020198	5.205097	6.271783e-09	0.00025	
10	10	2.000314	5.059252	4.430000e-09	0.00025	
11	11	2.000283	5.050460	5.060000e-09	0.00025	
12	12	2.000331	5.001894	5.740000e-09	0.00025	
13	13	2.000308	4.968804	5.760000e-09	0.00025	
14	14	2.000310	4.968385	5.470000e-09	0.00025	
15	15	2.000318	4.944184	4.840000e-09	0.00025	
16	16	2.000333	4.935716	4.580000e-09	0.00025	
17	17	2.000312	4.947436	5.640000e-09	0.00025	
18	18	2.000289	4.952896	5.910000e-09	0.00025	

	mean_current	std_current	max_slope	mean_slope	knee_voltage	curve_area
0	0.000074	0.000109	0.001440	0.000035	4.955229	0.000039
1	0.000071	0.000107	NaN	NaN	5.026642	0.000039
2	0.000070	0.000107	NaN	NaN	5.049632	0.000038
3	0.000067	0.000105	NaN	NaN	5.084735	0.000039
4	0.000067	0.000105	0.001420	0.000057	5.075762	0.000039
5	0.000065	0.000104	NaN	NaN	5.131911	0.000039
6	0.000068	0.000106	NaN	NaN	5.073487	0.000039
7	0.000065	0.000104	NaN	NaN	5.117414	0.000039
8	0.000079	0.000111	NaN	NaN	4.898752	0.000039
9	0.000061	0.000101	0.001409	0.000069	5.196236	0.000042
10	0.000069	0.000106	NaN	NaN	5.058979	0.000038
11	0.000069	0.000106	NaN	NaN	5.050294	0.000039
12	0.000072	0.000108	NaN	NaN	5.001660	0.000039
13	0.000074	0.000109	NaN	NaN	4.968307	0.000039
14	0.000074	0.000109	NaN	NaN	4.968085	0.000039
15	0.000076	0.000110	NaN	NaN	4.944067	0.000039
16	0.000076	0.000110	NaN	NaN	4.935489	0.000039
17	0.000076	0.000110	0.001493	0.000057	4.934627	0.000039

```

18      0.000075      0.000109      NaN      NaN      4.952729      0.000039
/usr/local/lib/python3.13/dist-packages/numpy/lib/_function_base_impl.py:1307: RuntimeWarning: divide by zero encountered in divide
a = -(dx2)/(dx1 * (dx1 + dx2))
/usr/local/lib/python3.13/dist-packages/numpy/lib/_function_base_impl.py:1308: RuntimeWarning: divide by zero encountered in divide
b = (dx2 - dx1) / (dx1 * dx2)
/usr/local/lib/python3.13/dist-packages/numpy/lib/_function_base_impl.py:1309: RuntimeWarning: divide by zero encountered in divide
c = dx1 / (dx2 * (dx1 + dx2))
/usr/local/lib/python3.13/dist-packages/numpy/lib/_function_base_impl.py:1315: RuntimeWarning: invalid value encountered in add
out[tuple(slice1)] = a * f[tuple(slice2)] + b * f[tuple(slice3)] + c * f[tuple(slice4)]
/usr/local/lib/python3.13/dist-packages/numpy/lib/_function_base_impl.py:1307: RuntimeWarning: invalid value encountered in divide
a = -(dx2)/(dx1 * (dx1 + dx2))
/usr/local/lib/python3.13/dist-packages/numpy/lib/_function_base_impl.py:1308: RuntimeWarning: invalid value encountered in divide
b = (dx2 - dx1) / (dx1 * dx2)
/usr/local/lib/python3.13/dist-packages/numpy/lib/_function_base_impl.py:1309: RuntimeWarning: invalid value encountered in divide
c = dx1 / (dx2 * (dx1 + dx2))

```

```

leakage_features = extract_curve_features(
    leakage_curves,
    "Applied_Voltage",
    "Leakage_Current"
)

```

```
print(leakage_features)
```

3	3	1.009896	600.0503	4.310000e-09	0.000004
4	4	1.007762	600.0496	4.230000e-09	0.000004
5	5	1.008198	600.0498	3.100000e-09	0.000004
6	6	1.008744	600.0488	3.400000e-09	0.000004
7	7	1.007158	600.0483	4.200000e-09	0.000004
8	8	1.008414	600.0482	4.480000e-09	0.000004
9	9	4.013724	600.0485	1.179669e-08	0.000005
10	10	4.014379	600.0499	1.056408e-08	0.000005
11	11	4.013053	600.0497	9.840659e-09	0.000005
12	12	4.015897	600.0491	9.830220e-09	0.000005
13	13	4.015024	600.0501	1.094856e-08	0.000005
14	14	4.012281	600.0515	1.358145e-08	0.000006
15	15	4.015242	600.0506	1.192419e-08	0.000006
16	16	4.014414	600.0486	1.171502e-08	0.000005
17	17	4.014631	600.0508	1.702018e-08	0.000005
18	18	4.014134	600.0509	1.636147e-08	0.000004

	mean_current	std_current	max_slope	mean_slope	knee_voltage \
0	0.000002	0.000001	1.488238e-08	7.904448e-09	590.9866
1	0.000002	0.000001	1.496338e-08	7.869943e-09	597.0400
2	0.000002	0.000001	1.333918e-08	7.474571e-09	587.9977
3	0.000002	0.000001	1.334551e-08	7.468299e-09	566.9396
4	0.000002	0.000001	1.334460e-08	7.324789e-09	566.9388
5	0.000002	0.000001	1.333907e-08	7.152069e-09	554.8902
6	0.000002	0.000001	1.334643e-08	7.092202e-09	566.9385
7	0.000002	0.000001	1.334118e-08	7.378158e-09	566.9372
8	0.000002	0.000001	1.334256e-08	7.493420e-09	566.9387
9	0.000002	0.000001	1.363325e-08	7.566136e-09	600.0485
10	0.000002	0.000001	1.384551e-08	7.725809e-09	600.0499
11	0.000002	0.000001	1.410673e-08	7.866972e-09	600.0497
12	0.000002	0.000001	1.460023e-08	8.162445e-09	600.0491
13	0.000002	0.000001	1.529594e-08	8.601588e-09	600.0501
14	0.000002	0.000002	1.654469e-08	9.458719e-09	600.0515
15	0.000002	0.000002	1.638815e-08	9.426688e-09	600.0506
16	0.000002	0.000002	1.547693e-08	8.898155e-09	600.0486
17	0.000002	0.000001	1.399688e-08	7.665374e-09	600.0508
18	0.000002	0.000001	1.391909e-08	7.464740e-09	600.0509

	curve_area
0	0.001102
1	0.001101
2	0.001036
3	0.001039
4	0.001016
5	0.000991
6	0.000983
7	0.001025
8	0.001043
9	0.001049
10	0.001075
11	0.001092
12	0.001133
13	0.001201
14	0.001336
15	0.001335
16	0.001254
17	0.001070
18	0.001034

```

turnon_features = extract_curve_features(
    turnon_curves,
    "Gate_Voltage",
    "Collector_Current"
)

```

```
print(turnon_features)
```

	curve_id	min_voltage	max_voltage	min_current	max_current \
0	0	2.020181	4.974007	5.885971e-09	0.00025
1	1	2.020185	5.026770	5.937019e-09	0.00025
2	2	2.020205	5.049754	6.093055e-09	0.00025
3	3	2.020159	5.084847	6.105468e-09	0.00025
4	4	2.020186	5.097120	6.623396e-09	0.00025
5	5	2.020182	5.132773	7.062614e-09	0.00025
6	6	2.020198	5.074069	6.591616e-09	0.00025
7	7	2.020194	5.117679	6.587623e-09	0.00025
8	8	2.020174	4.899527	6.104020e-09	0.00025
9	9	2.020198	5.205097	6.271783e-09	0.00025
10	10	2.000314	5.059252	4.430000e-09	0.00025
11	11	2.000283	5.050460	5.060000e-09	0.00025
12	12	2.000331	5.001894	5.740000e-09	0.00025
13	13	2.000308	4.968804	5.760000e-09	0.00025
14	14	2.000310	4.968385	5.470000e-09	0.00025
15	15	2.000318	4.944184	4.840000e-09	0.00025
16	16	2.000333	4.935716	4.580000e-09	0.00025
17	17	2.000312	4.947436	5.640000e-09	0.00025
18	18	2.000289	4.952896	5.910000e-09	0.00025

```

    mean_current std_current max_slope mean_slope knee_voltage curve_area
0      0.000074  0.000109  0.001440  0.000035  4.955229  0.000039
1      0.000071  0.000107      NaN      NaN  5.026642  0.000039
2      0.000070  0.000107      NaN      NaN  5.049632  0.000038
3      0.000067  0.000105      NaN      NaN  5.084735  0.000039
4      0.000067  0.000105  0.001420  0.000057  5.075762  0.000039
5      0.000065  0.000104      NaN      NaN  5.131911  0.000039
6      0.000068  0.000106      NaN      NaN  5.073487  0.000039
7      0.000065  0.000104      NaN      NaN  5.117414  0.000039
8      0.000079  0.000111      NaN      NaN  4.898752  0.000039
9      0.000061  0.000101  0.001409  0.000069  5.196236  0.000042
10     0.000069  0.000106      NaN      NaN  5.058979  0.000038
11     0.000069  0.000106      NaN      NaN  5.050294  0.000039
12     0.000072  0.000108      NaN      NaN  5.001660  0.000039
13     0.000074  0.000109      NaN      NaN  4.968307  0.000039
14     0.000074  0.000109      NaN      NaN  4.968085  0.000039
15     0.000076  0.000110      NaN      NaN  4.944067  0.000039
16     0.000076  0.000110      NaN      NaN  4.935489  0.000039
17     0.000076  0.000110  0.001493  0.000057  4.934627  0.000039
18     0.000075  0.000109      NaN      NaN  4.952729  0.000039
/usr/local/lib/python3.13/dist-packages/numpy/lib/_function_base_impl.py:1307: RuntimeWarning: divide by zero encountered in divide
a = -(dx2)/(dx1 * (dx1 + dx2))
/usr/local/lib/python3.13/dist-packages/numpy/lib/_function_base_impl.py:1308: RuntimeWarning: divide by zero encountered in divide
b = (dx2 - dx1) / (dx1 * dx2)
/usr/local/lib/python3.13/dist-packages/numpy/lib/_function_base_impl.py:1309: RuntimeWarning: divide by zero encountered in divide
c = dx1 / (dx2 * (dx1 + dx2))
/usr/local/lib/python3.13/dist-packages/numpy/lib/_function_base_impl.py:1315: RuntimeWarning: invalid value encountered in add
out[tuple(slice1)] = a * f[tuple(slice2)] + b * f[tuple(slice3)] + c * f[tuple(slice4)]
/usr/local/lib/python3.13/dist-packages/numpy/lib/_function_base_impl.py:1307: RuntimeWarning: invalid value encountered in divide
a = -(dx2)/(dx1 * (dx1 + dx2))
/usr/local/lib/python3.13/dist-packages/numpy/lib/_function_base_impl.py:1308: RuntimeWarning: invalid value encountered in divide
b = (dx2 - dx1) / (dx1 * dx2)
/usr/local/lib/python3.13/dist-packages/numpy/lib/_function_base_impl.py:1309: RuntimeWarning: invalid value encountered in divide
c = dx1 / (dx2 * (dx1 + dx2))

```

```

print("\nBREAKDOWN FEATURE STATISTICS")
print(breakdown_features.describe())

```

```

print("\nLEAKAGE FEATURE STATISTICS")
print(leakage_features.describe())

```

```

print("\nTURN-ON FEATURE STATISTICS")
print(turnon_features.describe())

```

```

mean      9.000000      2.590283      600.049595      8.454343e-09      4.723843e-06
std       5.627314      1.542154      0.000946      4.618507e-09      4.183326e-07
min       0.000000      1.007158      600.048200      3.100000e-09      4.250000e-06
25%      4.500000      1.007980      600.048800      4.215000e-09      4.473057e-06
50%      9.000000      4.012281      600.049600      9.830220e-09      4.586193e-06
75%     13.500000      4.014397      600.050200      1.175585e-08      4.807104e-06
max     18.000000      4.015897      600.051500      1.702018e-08      5.649598e-06

```

```

      mean_current std_current max_slope mean_slope knee_voltage \
count  1.900000e+01  1.900000e+01  1.900000e+01  1.900000e+01  19.000000
mean   1.845183e-06  1.369472e-06  1.426588e-08  7.894449e-09  587.689842
std    1.811515e-07  1.214526e-07  1.053155e-09  7.091984e-10  16.423242
min    1.642571e-06  1.234458e-06  1.333907e-08  7.092202e-09  554.890200
25%    1.734551e-06  1.293363e-06  1.334506e-08  7.466519e-09  566.939200
50%    1.797018e-06  1.322681e-06  1.391909e-08  7.665374e-09  600.048500
75%    1.873213e-06  1.395533e-06  1.492288e-08  8.033447e-09  600.050000
max    2.244281e-06  1.638416e-06  1.654469e-08  9.458719e-09  600.051500

```

```

      curve_area
count  19.000000
mean   0.001101
std    0.000106
min    0.000983
25%    0.001035
50%    0.001070
75%    0.001118
max    0.001336

```

TURN-ON FEATURE STATISTICS

```

      curve_id min_voltage max_voltage min_current max_current \
count  19.000000  19.000000  19.000000  1.900000e+01  1.900000e+01
mean   9.000000    2.010772    5.025825  5.825924e-09  2.500249e-04
std    5.627314    0.010196    0.080696  7.076203e-10  3.477581e-10
min    0.000000    2.000283    4.899527  4.430000e-09  2.500240e-04
25%    4.500000    2.000313    4.960641  5.555000e-09  2.500248e-04
50%    9.000000    2.020159    5.026770  5.910000e-09  2.500250e-04
75%   13.500000    2.020186    5.079458  6.188625e-09  2.500250e-04
max   18.000000    2.020205    5.205097  7.062614e-09  2.500254e-04

```

```

      mean_current std_current max_slope mean_slope knee_voltage \
count  19.000000  19.000000  4.000000  4.000000  19.000000
mean   0.000071  0.000107  0.001440  0.000055  5.022318
std    0.000005  0.000003  0.000038  0.000014  0.080212
min    0.000061  0.000101  0.001409  0.000035  4.898752
25%    0.000068  0.000106  0.001417  0.000051  4.953979
50%    0.000071  0.000107  0.001430  0.000057  5.026642
75%    0.000075  0.000109  0.001453  0.000060  5.074625
max    0.000079  0.000111  0.001493  0.000069  5.196236

```

```

      curve_area
count  1.900000e+01
mean   3.885062e-05
std    7.803628e-07
min    3.847716e-05
25%    3.857043e-05
50%    3.867691e-05
75%    3.871449e-05
max    4.200073e-05

```

```

print("\nBreakdown:")
print(breakdown_features)

```

```

print("\nLeakage:")
print(leakage_features)

```

```
print("\nTurn-On:")
print(turnon_features)
```

Breakdown:

	curve_id	min_voltage	max_voltage	min_current	max_current	\
0	0	2.020181	4.974007	5.885971e-09	0.00025	
1	1	2.020185	5.026770	5.937019e-09	0.00025	
2	2	2.020205	5.049754	6.093055e-09	0.00025	
3	3	2.020159	5.084847	6.105468e-09	0.00025	
4	4	2.020186	5.097120	6.623396e-09	0.00025	
5	5	2.020182	5.132773	7.062614e-09	0.00025	
6	6	2.020198	5.074069	6.591616e-09	0.00025	
7	7	2.020194	5.117679	6.587623e-09	0.00025	
8	8	2.020174	4.899527	6.104020e-09	0.00025	
9	9	2.020198	5.205097	6.271783e-09	0.00025	
10	10	2.000314	5.059252	4.430000e-09	0.00025	
11	11	2.000283	5.050460	5.060000e-09	0.00025	
12	12	2.000331	5.001894	5.740000e-09	0.00025	
13	13	2.000308	4.968804	5.760000e-09	0.00025	
14	14	2.000310	4.968385	5.470000e-09	0.00025	
15	15	2.000318	4.944184	4.840000e-09	0.00025	
16	16	2.000333	4.935716	4.580000e-09	0.00025	
17	17	2.000312	4.947436	5.640000e-09	0.00025	
18	18	2.000289	4.952896	5.910000e-09	0.00025	

	mean_current	std_current	max_slope	mean_slope	knee_voltage	curve_area
0	0.000074	0.000109	0.001440	0.000035	4.955229	0.000039
1	0.000071	0.000107	NaN	NaN	5.026642	0.000039
2	0.000070	0.000107	NaN	NaN	5.049632	0.000038
3	0.000067	0.000105	NaN	NaN	5.084735	0.000039
4	0.000067	0.000105	0.001420	0.000057	5.075762	0.000039
5	0.000065	0.000104	NaN	NaN	5.131911	0.000039
6	0.000068	0.000106	NaN	NaN	5.073487	0.000039
7	0.000065	0.000104	NaN	NaN	5.117414	0.000039
8	0.000079	0.000111	NaN	NaN	4.898752	0.000039
9	0.000061	0.000101	0.001409	0.000069	5.196236	0.000042
10	0.000069	0.000106	NaN	NaN	5.058979	0.000038
11	0.000069	0.000106	NaN	NaN	5.050294	0.000039
12	0.000072	0.000108	NaN	NaN	5.001660	0.000039
13	0.000074	0.000109	NaN	NaN	4.968307	0.000039
14	0.000074	0.000109	NaN	NaN	4.968085	0.000039
15	0.000076	0.000110	NaN	NaN	4.944067	0.000039
16	0.000076	0.000110	NaN	NaN	4.935489	0.000039
17	0.000076	0.000110	0.001493	0.000057	4.934627	0.000039
18	0.000075	0.000109	NaN	NaN	4.952729	0.000039

Leakage:

	curve_id	min_voltage	max_voltage	min_current	max_current	\
0	0	1.007554	600.0488	3.850000e-09	0.000005	
1	1	1.007328	600.0493	5.610000e-09	0.000005	
2	2	1.007544	600.0495	3.870000e-09	0.000004	
3	3	1.009896	600.0503	4.310000e-09	0.000004	
4	4	1.007762	600.0496	4.230000e-09	0.000004	
5	5	1.008198	600.0498	3.100000e-09	0.000004	
6	6	1.008744	600.0488	3.400000e-09	0.000004	
7	7	1.007158	600.0483	4.200000e-09	0.000004	
8	8	1.008414	600.0482	4.480000e-09	0.000004	
9	9	4.013724	600.0485	1.179669e-08	0.000005	
10	10	4.014379	600.0499	1.056408e-08	0.000005	
11	11	4.013053	600.0497	9.840659e-09	0.000005	

```
import numpy as np
import pandas as pd

def extract_curve_features_v2(df, voltage_col, current_col):

    features = []

    for curve_id, curve in df.groupby("curve_id"):

        curve = curve[[voltage_col, current_col]].copy()

        # Remove missing values
        curve = curve.dropna()

        # Sort by voltage
        curve = curve.sort_values(voltage_col)

        voltage = curve[voltage_col].to_numpy()
        current = curve[current_col].to_numpy()

        # -----
        # Remove duplicate voltage values
        # -----

        temp = pd.DataFrame({
            "voltage": voltage,
            "current": current
        })

        temp = (
            temp
            .groupby("voltage", as_index=False)["current"]
            .mean()
            .sort_values("voltage")
        )

        voltage = temp["voltage"].to_numpy()
        current = temp["current"].to_numpy()

        # -----
        # Basic features
        # -----

        min_voltage = voltage.min()
        max_voltage = voltage.max()
```

```

min_current = current.min()
max_current = current.max()

mean_current = current.mean()
std_current = current.std()

# -----
# Safe slope calculation
# -----

dv = np.diff(voltage)
di = np.diff(current)

valid = dv > 1e-12

slopes = di[valid] / dv[valid]

if len(slopes) > 0:
    max_slope = np.max(slopes)
    mean_slope = np.mean(slopes)

    slope_index = np.argmax(
        slopes
    )

    knee_voltage = voltage[
        np.where(valid)[0][slope_index]
    ]

else:
    max_slope = np.nan
    mean_slope = np.nan
    knee_voltage = np.nan

# -----
# Current at different voltage regions
# -----

v25 = min_voltage + 0.25 * (
    max_voltage - min_voltage
)

v50 = min_voltage + 0.50 * (
    max_voltage - min_voltage
)

v75 = min_voltage + 0.75 * (
    max_voltage - min_voltage
)

v90 = min_voltage + 0.90 * (
    max_voltage - min_voltage
)

current_v25 = np.interp(
    v25, voltage, current
)

current_v50 = np.interp(
    v50, voltage, current
)

current_v75 = np.interp(
    v75, voltage, current
)

current_v90 = np.interp(
    v90, voltage, current
)

# -----
# Current-normalized knee points
# Useful for breakdown / turn-on
# -----

threshold_10 = 0.10 * max_current
threshold_50 = 0.50 * max_current
threshold_90 = 0.90 * max_current

idx10 = np.argmax(current >= threshold_10)
idx50 = np.argmax(current >= threshold_50)
idx90 = np.argmax(current >= threshold_90)

voltage_10pct = voltage[idx10]
voltage_50pct = voltage[idx50]
voltage_90pct = voltage[idx90]

# -----
# Area under curve
# -----

curve_area = np.trapezoid(
    current,
    voltage
)

# -----
# Store
# -----

features.append({
    "curve_id": curve_id,

```



```

        "min_voltage": min_voltage,
        "max_voltage": max_voltage,

        "min_current": min_current,
        "max_current": max_current,

        "mean_current": mean_current,
        "std_current": std_current,

        "max_slope": max_slope,
        "mean_slope": mean_slope,

        "knee_voltage": knee_voltage,

        "current_v25": current_v25,
        "current_v50": current_v50,
        "current_v75": current_v75,
        "current_v90": current_v90,

        "voltage_10pct": voltage_10pct,
        "voltage_50pct": voltage_50pct,
        "voltage_90pct": voltage_90pct,

        "curve_area": curve_area
    })

    return pd.DataFrame(features)

```

```

breakdown_features = extract_curve_features_v2(
    breakdown_curves,
    "Collector_Emitter_Voltage_Vce",
    "Leakage_Current_Ic"
)

leakage_features = extract_curve_features_v2(
    leakage_curves,
    "Applied_Voltage",
    "Leakage_Current"
)

turnon_features = extract_curve_features_v2(
    turnon_curves,
    "Gate_Voltage",
    "Collector_Current"
)

print("Breakdown:", breakdown_features.shape)
print("Leakage:", leakage_features.shape)
print("Turn-On:", turnon_features.shape)

```

```

Breakdown: (19, 18)
Leakage: (19, 18)
Turn-On: (19, 18)

```

```

print("Breakdown NaNs:")
print(breakdown_features.isna().sum())

print("\nLeakage NaNs:")
print(leakage_features.isna().sum())

print("\nTurn-On NaNs:")
print(turnon_features.isna().sum())

```

```

Breakdown NaNs:
curve_id      0
min_voltage   0
max_voltage   0
min_current   0
max_current   0
mean_current   0
std_current   0
max_slope     0
mean_slope    0
knee_voltage   0
current_v25    0
current_v50    0
current_v75    0
current_v90    0
voltage_10pct  0
voltage_50pct  0
voltage_90pct  0
curve_area     0
dtype: int64

```

```

Leakage NaNs:
curve_id      0
min_voltage   0
max_voltage   0
min_current   0
max_current   0
mean_current   0
std_current   0
max_slope     0
mean_slope    0
knee_voltage   0
current_v25    0
current_v50    0
current_v75    0
current_v90    0
voltage_10pct  0
voltage_50pct  0
voltage_90pct  0
curve_area     0
dtype: int64

```

```
Turn-On NaNs:
curve_id      0
min_voltage   0
max_voltage   0
min_current   0
max_current   0
mean_current  0
std_current   0
max_slope     0
mean_slope    0
knee_voltage  0
current_v25   0
current_v50   0
current_v75   0
current_v90   0
```

```
import matplotlib.pyplot as plt

feature_columns = [
    "mean_current",
    "std_current",
    "max_slope",
    "mean_slope",
    "knee_voltage",
    "current_v25",
    "current_v50",
    "current_v75",
    "current_v90",
    "voltage_10pct",
    "voltage_50pct",
    "voltage_90pct",
    "curve_area"
]

fig, axes = plt.subplots(4, 4, figsize=(16, 14))

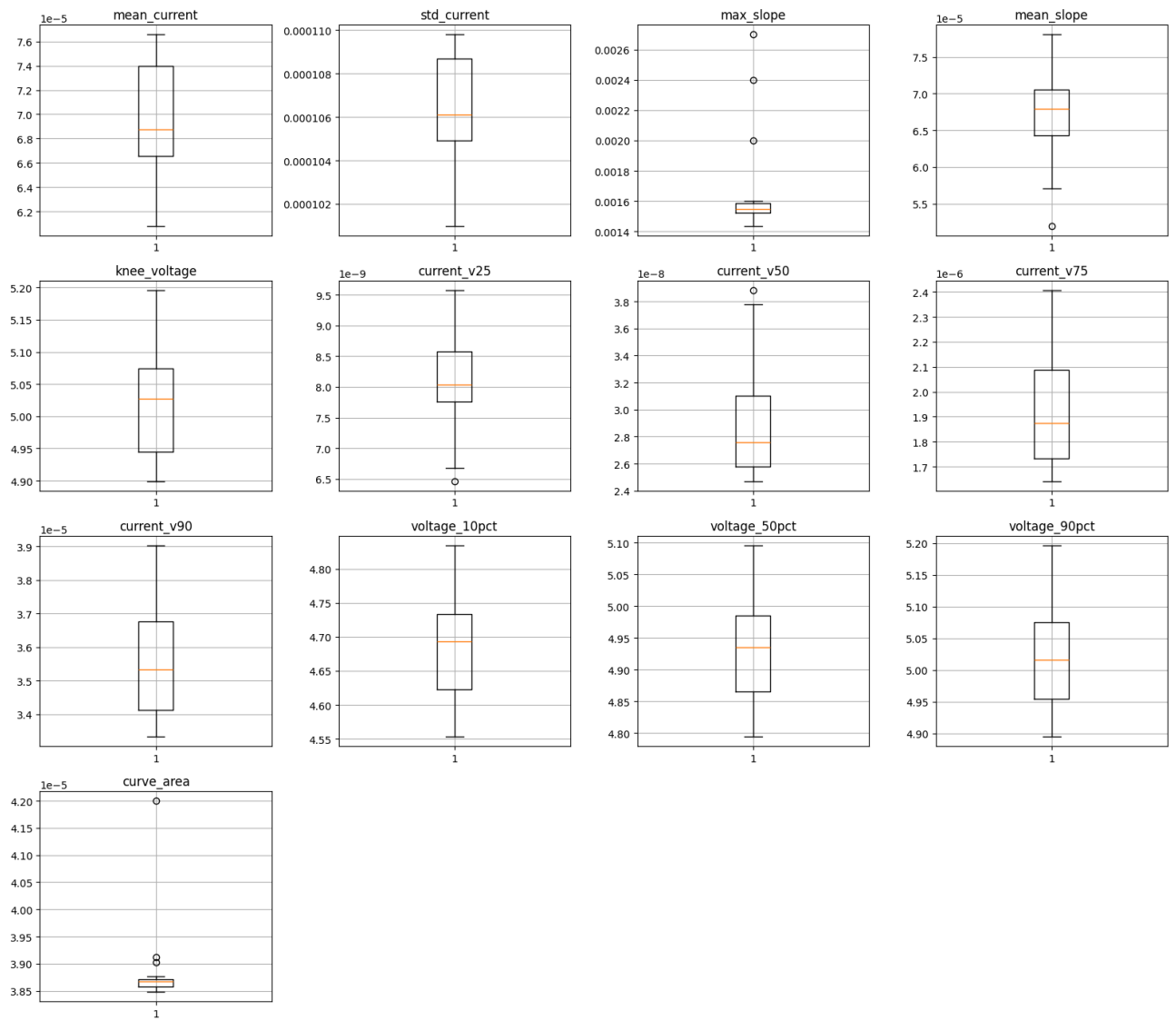
axes = axes.flatten()

for i, feature in enumerate(feature_columns):
    axes[i].boxplot(
        breakdown_features[feature]
    )

    axes[i].set_title(feature)
    axes[i].grid(True)

# Hide unused plots
for j in range(len(feature_columns), len(axes)):
    axes[j].axis("off")

plt.tight_layout()
plt.show()
```



```
fig, axes = plt.subplots(4, 4, figsize=(16, 14))

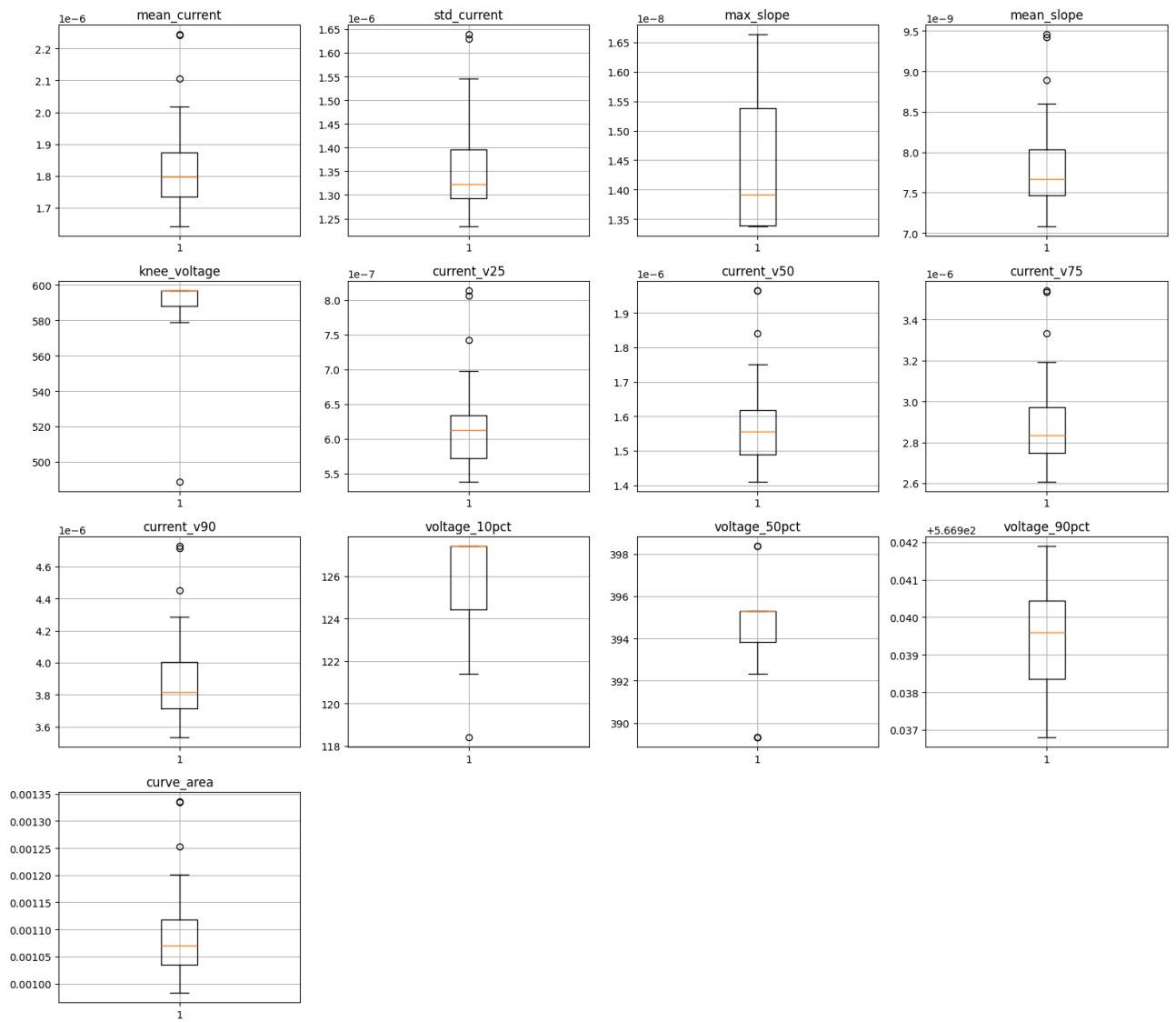
axes = axes.flatten()

for i, feature in enumerate(feature_columns):
    axes[i].boxplot(
        leakage_features[feature]
    )

    axes[i].set_title(feature)
    axes[i].grid(True)

for j in range(len(feature_columns), len(axes)):
    axes[j].axis("off")

plt.tight_layout()
plt.show()
```



```
fig, axes = plt.subplots(4, 4, figsize=(16, 14))

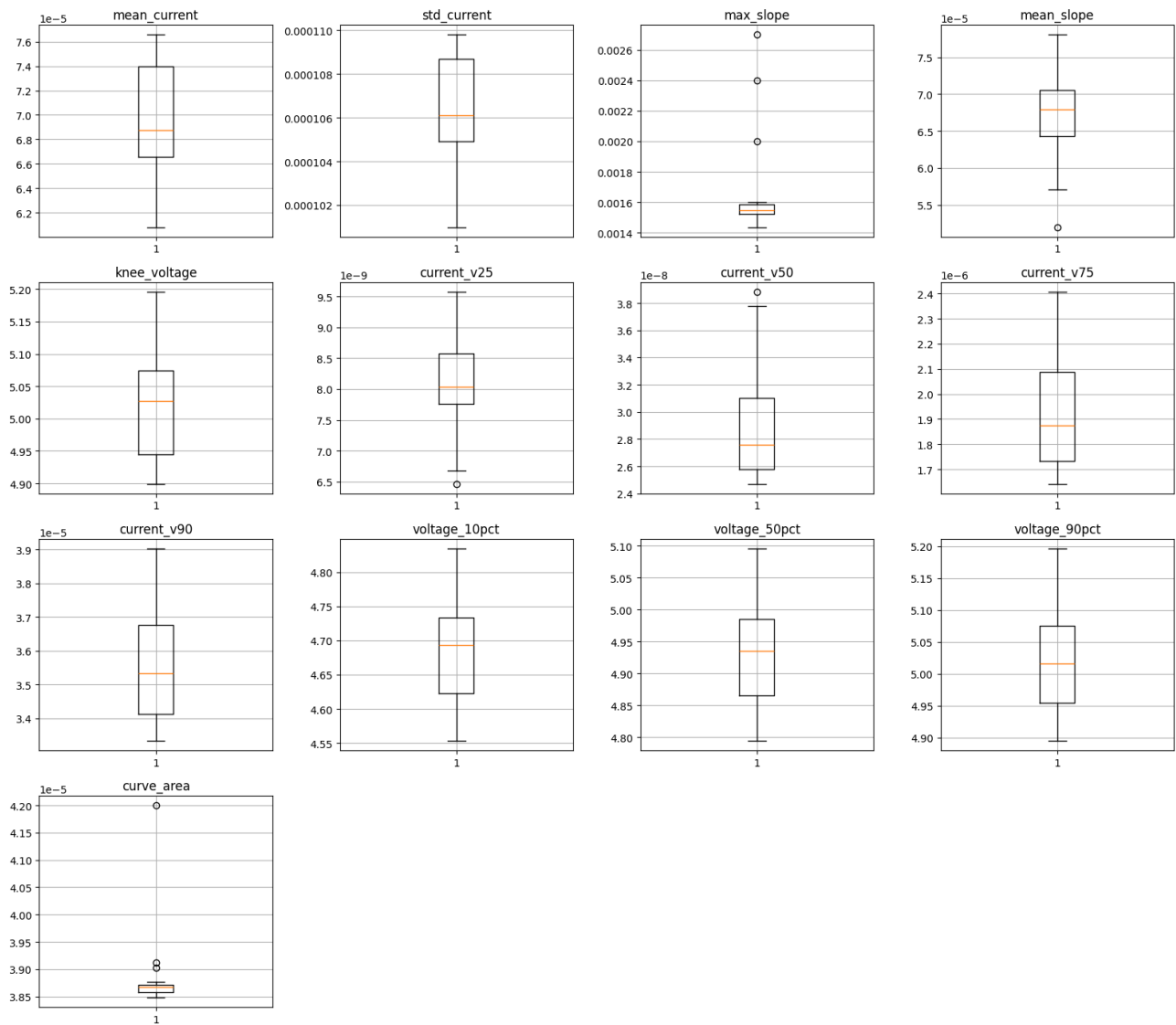
axes = axes.flatten()

for i, feature in enumerate(feature_columns):
    axes[i].boxplot(
        turnon_features[feature]
    )

    axes[i].set_title(feature)
    axes[i].grid(True)

for j in range(len(feature_columns), len(axes)):
    axes[j].axis("off")

plt.tight_layout()
plt.show()
```



```
import numpy as np
import pandas as pd

from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import IsolationForest
```

```
feature_columns = [
    "min_voltage",
    "max_voltage",
    "min_current",
    "max_current",
    "mean_current",
    "std_current",
    "max_slope",
    "mean_slope",
    "knee_voltage",
    "current_v25",
    "current_v50",
    "current_v75",
    "current_v90",
    "voltage_10pct",
    "voltage_50pct",
    "voltage_90pct",
    "curve_area"
]

X_breakdown = breakdown_features[feature_columns].copy()

print("Feature matrix shape:", X_breakdown.shape)
print(X_breakdown.head())
```

```
Feature matrix shape: (19, 17)
  min_voltage  max_voltage  min_current  max_current  mean_current \
0    2.020181    4.974007  5.885971e-09    0.00025    0.000074
1    2.020185    5.026770  5.937019e-09    0.00025    0.000067
2    2.020205    5.049754  6.093055e-09    0.00025    0.000069
3    2.020159    5.084847  6.105468e-09    0.00025    0.000065
4    2.020186    5.097120  6.623396e-09    0.00025    0.000067

  std_current  max_slope  mean_slope  knee_voltage  current_v25 \
0    0.000109    0.001522    0.000052    4.955229  7.921240e-09
1    0.000105    0.001600    0.000071    5.026754  8.032987e-09
2    0.000106    0.002700    0.000078    5.049729  8.095472e-09
3    0.000104    0.001556    0.000068    5.075696  8.222264e-09
```

4	0.000105	0.001600	0.000065	5.096733	8.823758e-09
---	----------	----------	----------	----------	--------------

	current_v50	current_v75	current_v90	voltage_10pct	voltage_50pct \
0	3.086347e-08	0.000002	0.000037	4.633350	4.874881
1	2.757866e-08	0.000002	0.000035	4.693780	4.934626
2	2.623061e-08	0.000002	0.000035	4.714026	4.955209
3	2.519135e-08	0.000002	0.000034	4.733964	4.995436
4	2.577250e-08	0.000002	0.000034	4.754558	4.995391

	voltage_90pct	curve_area
0	4.973337	0.000039
1	5.015338	0.000039
2	5.035233	0.000038
3	5.075696	0.000039
4	5.095985	0.000039

```

scaler = StandardScaler()

X_scaled = scaler.fit_transform(X_breakdown)

print("Scaled shape:", X_scaled.shape)
print("Mean of first feature:", X_scaled[:, 0].mean())
print("Std of first feature:", X_scaled[:, 0].std())

```

```

Scaled shape: (19, 17)
Mean of first feature: -3.7700836604639525e-14
Std of first feature: 1.0

```

```

def robust_iqr_score(series):
    median = series.median()
    q1 = series.quantile(0.25)
    q3 = series.quantile(0.75)

    iqr = q3 - q1

    # If feature has essentially no variation
    if iqr == 0:
        return pd.Series(
            np.zeros(len(series)),
            index=series.index
        )

    # Robust z-score based on IQR
    return (series - median) / (iqr / 1.349)

# Calculate dynamic scores for every feature
robust_scores = pd.DataFrame(index=breakdown_features.index)

for col in feature_columns:
    robust_scores[col] = robust_iqr_score(
        breakdown_features[col]
    )

print(robust_scores.round(3))

```

3	-0.831	0.195	0.112	0.509	0.312
4	-0.375	1.164	-0.627	0.727	1.305
5	-1.037	-0.685	0.000	0.926	2.557
6	-0.479	18.838	-0.598	0.489	1.537
7	-1.262	-0.792	1.336	0.926	1.776
8	1.317	10.001	1.387	-1.326	1.179
9	-1.824	-2.442	0.532	1.761	0.621
10	-0.165	-0.061	0.082	0.301	0.485
11	-0.061	-0.235	-1.893	0.088	-0.046
12	0.324	0.457	-0.124	-0.326	-0.632
13	0.693	-0.070	0.587	-0.744	-0.397
14	0.177	0.036	-1.507	-0.743	-0.757
15	0.951	0.097	-2.356	-0.958	-2.233
16	0.892	-0.700	-0.458	-1.164	-2.595
17	1.228	0.000	-0.953	-0.957	-0.501
18	1.028	-0.508	0.386	-0.957	-0.020

	current_v50	current_v75	current_v90	voltage_10pct	voltage_50pct \
0	0.850	0.715	0.678	-0.737	-0.669
1	0.000	-0.029	-0.080	0.000	0.000
2	-0.349	-0.337	-0.391	0.247	0.230
3	-0.618	-0.630	-0.742	0.490	0.681
4	-0.467	-0.665	-0.776	0.741	0.680
5	-0.453	-0.762	-0.817	1.227	1.126
6	-0.179	-0.455	-0.507	0.490	0.454
7	-0.602	-0.894	-1.030	0.983	0.903
8	2.915	2.034	1.892	-1.713	-1.571
9	2.643	1.853	0.745	1.717	1.806

```

14      -0.071      -0.300
15      -0.901      -0.754
16      -0.901      0.295
17      -0.901      0.805
18      -0.705      0.330

```

```

print("\nMaximum absolute dynamic score for each curve:")

dynamic_score = robust_scores.abs().max(axis=1)

print(
    pd.DataFrame({
        "curve_id": breakdown_features["curve_id"],
        "dynamic_score": dynamic_score
    }).round(3)
)

```

```

Maximum absolute dynamic score for each curve:
   curve_id  dynamic_score
0          0          3.449
1          1          1.619
2          2         25.466
3          3          3.238
4          4          1.619
5          5          4.141
6          6         18.838
7          7          4.856
8          8         10.001
9          9         31.123
10         10          3.151
11         11          1.893
12         12          1.398
13         13          1.348
14         14          1.507
15         15          2.356
16         16          2.832
17         17          5.396
18         18          1.349

```

▾ actual Dynamic Outlier Detector

```

# Threshold for an individual feature
FEATURE_THRESHOLD = 3.5

# Count how many features are unusually different
outlier_feature_count = (
    robust_scores.abs() >= FEATURE_THRESHOLD
).sum(axis=1)

dynamic_results = pd.DataFrame({
    "curve_id": breakdown_features["curve_id"],
    "dynamic_score": dynamic_score,
    "outlier_feature_count": outlier_feature_count
})

print(dynamic_results.sort_values(
    "dynamic_score",
    ascending=False
).round(3))

```

```

   curve_id  dynamic_score  outlier_feature_count
9          9         31.123                      1
2          2         25.466                      1
6          6         18.838                      1
8          8         10.001                      1
17         17          5.396                      1
7          7          4.856                      1
5          5          4.141                      1
0          0          3.449                      0
3          3          3.238                      0
10         10          3.151                      0
16         16          2.832                      0
15         15          2.356                      0
11         11          1.893                      0
1          1          1.619                      0
4          4          1.619                      0
14         14          1.507                      0
12         12          1.398                      0
18         18          1.349                      0
13         13          1.348                      0

```

```

for idx, row in dynamic_results.iterrows():

    unusual_features = robust_scores.columns[
        robust_scores.loc[idx].abs() >= FEATURE_THRESHOLD
    ].tolist()

    if len(unusual_features) > 0:
        print(
            f"\nCurve {int(row['curve_id'])}:"
        )
        print(
            "Unusual features:",
            unusual_features
        )

```

```

Curve 2:
Unusual features: ['max_slope']

Curve 5:
Unusual features: ['curve_area']

Curve 6:

```

```

Unusual features: ['max_slope']

Curve 7:
Unusual features: ['max_current']

Curve 8:
Unusual features: ['max_slope']

Curve 9:
Unusual features: ['curve_area']

Curve 17:
Unusual features: ['max_current']

```

Isolation Forest

```

from sklearn.ensemble import IsolationForest

# Create Isolation Forest model
isolation_forest = IsolationForest(
    n_estimators=300,
    contamination=0.10,
    random_state=42
)

# Train the model and predict anomalies
isolation_predictions = isolation_forest.fit_predict(X_scaled)

# Convert Isolation Forest scores so that
# higher = more anomalous
isolation_score = -isolation_forest.score_samples(X_scaled)

# Convert predictions to readable labels
ml_flag = np.where(
    isolation_predictions == -1,
    "ANOMALY",
    "NORMAL"
)

# Store results
ml_results = pd.DataFrame({
    "curve_id": breakdown_features["curve_id"],
    "isolation_score": isolation_score,
    "ml_flag": ml_flag
})

print(ml_results.sort_values(
    "isolation_score",
    ascending=False
).round(4))

```

	curve_id	isolation_score	ml_flag
9	9	0.6306	ANOMALY
8	8	0.6185	ANOMALY
5	5	0.5308	NORMAL
7	7	0.5169	NORMAL
16	16	0.5025	NORMAL
17	17	0.4776	NORMAL
2	2	0.4771	NORMAL
15	15	0.4742	NORMAL
10	10	0.4714	NORMAL
6	6	0.4598	NORMAL
0	0	0.4584	NORMAL
11	11	0.4552	NORMAL
4	4	0.4527	NORMAL
3	3	0.4486	NORMAL
12	12	0.4458	NORMAL
14	14	0.4301	NORMAL
1	1	0.4239	NORMAL
18	18	0.4223	NORMAL
13	13	0.4096	NORMAL

```

# Combine dynamic detector and Isolation Forest results

results = dynamic_results.merge(
    ml_results,
    on="curve_id"
)

# Final decision
results["final_decision"] = np.where(
    (results["outlier_feature_count"] >= 1) &
    (results["ml_flag"] == "ANOMALY"),
    "REJECT",
    np.where(
        (results["outlier_feature_count"] >= 1) |
        (results["ml_flag"] == "ANOMALY"),
        "HOLD",
        "PASS"
    )
)

# Sort by anomaly evidence
results = results.sort_values(
    ["final_decision", "dynamic_score"],
    ascending=[True, False]
)

print(results.round(4))

```

	curve_id	dynamic_score	outlier_feature_count	isolation_score	ml_flag	\
2	2	25.4660	1	0.4771	NORMAL	
6	6	18.8380	1	0.4598	NORMAL	
17	17	5.3960	1	0.4776	NORMAL	

7	7	4.8564	1	0.5169	NORMAL
5	5	4.1412	1	0.5308	NORMAL
0	0	3.4486	0	0.4584	NORMAL
3	3	3.2376	0	0.4486	NORMAL
10	10	3.1509	0	0.4714	NORMAL
16	16	2.8316	0	0.5025	NORMAL
15	15	2.3559	0	0.4742	NORMAL
11	11	1.8935	0	0.4552	NORMAL
1	1	1.6188	0	0.4239	NORMAL
4	4	1.6188	0	0.4527	NORMAL
14	14	1.5069	0	0.4301	NORMAL
12	12	1.3978	0	0.4458	NORMAL
18	18	1.3488	0	0.4223	NORMAL
13	13	1.3475	0	0.4096	NORMAL
9	9	31.1230	1	0.6306	ANOMALY
8	8	10.0008	1	0.6185	ANOMALY

```

final_decision
2      HOLD
6      HOLD
17     HOLD
7      HOLD
5      HOLD
0      PASS
3      PASS
10     PASS
16     PASS
15     PASS
11     PASS
1      PASS
4      PASS
14     PASS
12     PASS
18     PASS
13     PASS
9      REJECT
8      REJECT

```

```
# Show the feature values of the strongest anomalies
```

```

top_anomalies = results[
    results["final_decision"] == "REJECT"
]["curve_id"].tolist()

print("Strongest anomaly curves:", top_anomalies)

for curve_id in top_anomalies:

    row = breakdown_features[
        breakdown_features["curve_id"] == curve_id
    ].iloc[0]

    print(f"\n{'=' * 50}")
    print(f"CURVE {curve_id}")
    print(f"{'=' * 50}")

    for feature in feature_columns:
        print(f"{feature:20s}: {row[feature]:.8g}")

```

```
Strongest anomaly curves: [9, 8]
```

```

=====
CURVE 9
=====
min_voltage      : 2.020198
max_voltage      : 5.205097
min_current      : 6.271783e-09
max_current      : 0.0002500248
mean_current     : 6.0822678e-05
std_current      : 0.00010099538
max_slope        : 0.0014367876
mean_slope       : 7.038716e-05
knee_voltage     : 5.196236
current_v25      : 8.4094659e-09
current_v50      : 3.7794093e-08
current_v75      : 2.3607373e-06
current_v90      : 3.6798043e-05
voltage_10pct    : 4.834504
voltage_50pct    : 5.095993
voltage_90pct    : 5.196236
curve_area       : 4.2000728e-05

=====
CURVE 8
=====
min_voltage      : 2.020174
max_voltage      : 4.899527
min_current      : 6.10402e-09
max_current      : 0.0002500246
mean_current     : 7.6569829e-05
std_current      : 0.00010979426
max_slope        : 0.002
mean_slope       : 7.4340665e-05
knee_voltage     : 4.89913
current_v25      : 8.7470696e-09
current_v50      : 3.8847286e-08
current_v75      : 2.4080528e-06
current_v90      : 3.9037026e-05
voltage_10pct    : 4.553385
voltage_50pct    : 4.794242
voltage_90pct    : 4.89481
curve_area       : 3.9030358e-05

```

```

def generate_anomaly_report(curve_id):

    # Find the feature row for this curve
    feature_row = breakdown_features[
        breakdown_features["curve_id"] == curve_id
    ].iloc[0]

```

```

# Find the corresponding robust-score row
score_row = robust_scores.loc[
    breakdown_features["curve_id"] == curve_id
].iloc[0]

# Remove curve_id if it exists
score_row = score_row.drop(
    labels=["curve_id"],
    errors="ignore"
)

# Sort features by absolute robust score
scores = score_row.abs().sort_values(ascending=False)

print("=" * 60)
print(f"CURVE {curve_id} - ANOMALY REPORT")
print("=" * 60)

print("\nTop unusual features:")

for feature, score in scores.head(5).items():
    print(f"{feature:20s} → robust score = {score:.3f}")

print("\nActual feature values:")

for feature in scores.head(5).index:
    print(f"{feature:20s} → {feature_row[feature]:.8g}")

# Get dynamic score
dynamic_value = dynamic_results.loc[
    dynamic_results["curve_id"] == curve_id,
    "dynamic_score"
].iloc[0]

# Get Isolation Forest result
iso_row = ml_results[
    ml_results["curve_id"] == curve_id
].iloc[0]

iso_value = iso_row["isolation_score"]
ml_flag = iso_row["ml_flag"]

print("\nModel scores:")
print(f"Dynamic score      : {dynamic_value:.3f}")
print(f"Isolation score     : {iso_value:.4f}")
print(f"Isolation Forest    : {ml_flag}")

print("\nFinal interpretation:")

if dynamic_value >= 3.5 and ml_flag == "ANOMALY":
    print("🔴 STRONG POPULATION-RELATIVE ANOMALY")
elif dynamic_value >= 3.5:
    print("🟡 DYNAMIC OUTLIER")
elif ml_flag == "ANOMALY":
    print("🟡 ISOLATION FOREST ANOMALY")
else:
    print("🟢 NO STRONG ANOMALY")

```

```

generate_anomaly_report(8)
generate_anomaly_report(9)

```

```

=====
CURVE 8 - ANOMALY REPORT
=====

Top unusual features:
max_slope           → robust score = 10.001
curve_area          → robust score = 3.310
current_v50         → robust score = 2.915
max_current         → robust score = 2.158
current_v75         → robust score = 2.034

Actual feature values:
max_slope           → 0.002
curve_area          → 3.9030358e-05
current_v50         → 3.8847286e-08
max_current         → 0.0002500246
current_v75         → 2.4080528e-06

Model scores:
Dynamic score       : 10.001
Isolation score     : 0.6185
Isolation Forest    : ANOMALY

Final interpretation:
🔴 STRONG POPULATION-RELATIVE ANOMALY
=====
CURVE 9 - ANOMALY REPORT
=====

Top unusual features:
curve_area          → robust score = 31.123
current_v50         → robust score = 2.643
max_slope           → robust score = 2.442
max_voltage         → robust score = 2.025
voltage_90pct       → robust score = 2.019

Actual feature values:
curve_area          → 4.2000728e-05
current_v50         → 3.7794093e-08
max_slope           → 0.0014367876
max_voltage         → 5.205097
voltage_90pct       → 5.196236

Model scores:
Dynamic score       : 31.123

```

Isolation score : 0.6306
Isolation Forest : ANOMALY

Final interpretation:
● STRONG POPULATION-RELATIVE ANOMALY

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

def plot_anomalies(curves, x_col, y_col, anomaly_ids, title):

    plt.figure(figsize=(10, 6))

    # Plot normal curves
    for curve_id, group in curves.groupby("curve_id"):

        if curve_id not in anomaly_ids:
            plt.plot(
                group[x_col],
                group[y_col],
                linewidth=1,
                alpha=0.35
            )

    # Plot anomalies prominently
    for curve_id in anomaly_ids:

        group = curves[curves["curve_id"] == curve_id]

        plt.plot(
            group[x_col],
            group[y_col],
            linewidth=3,
            label=f"Anomaly Curve {curve_id}"
        )

    plt.xlabel(x_col)
    plt.ylabel(y_col)
    plt.title(title)
    plt.grid(True, alpha=0.3)
    plt.legend()
    plt.show()
```

```
dynamic_threshold = (
    dynamic_results["dynamic_score"].median()
    + 3 * (
        dynamic_results["dynamic_score"]
        - dynamic_results["dynamic_score"].median()
    ).abs().median()
)

print("Dynamic threshold:", round(dynamic_threshold, 3))

dynamic_outliers = dynamic_results[
    dynamic_results["dynamic_score"] >= dynamic_threshold
]

print("\nDynamic outliers:")
print(
    dynamic_outliers[
        ["curve_id", "dynamic_score"]
    ].sort_values(
        "dynamic_score",
        ascending=False
    ).round(3)
)
```

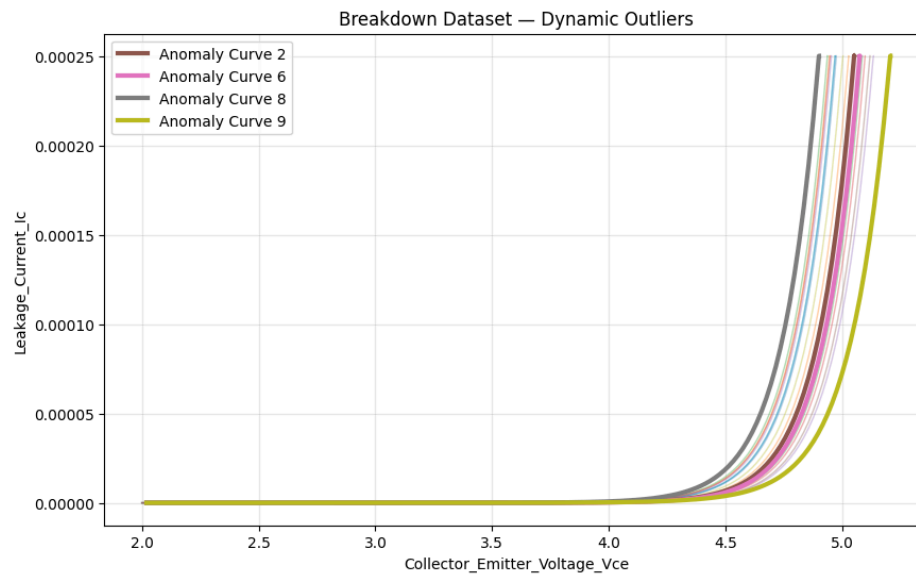
Dynamic threshold: 8.083

Dynamic outliers:

	curve_id	dynamic_score
9	9	31.123
2	2	25.466
6	6	18.838
8	8	10.001

```
breakdown_anomaly_ids = dynamic_outliers["curve_id"].tolist()

plot_anomalies(
    breakdown_curves,
    "Collector_Emitter_Voltage_Vce",
    "Leakage_Current_Ic",
    breakdown_anomaly_ids,
    "Breakdown Dataset - Dynamic Outliers"
)
```



```
thresholds = [2.5, 3.0, 3.5, 4.0, 5.0, 6.0, 7.0, 8.0, 10.0]
```

```
threshold_results = []
```

```
for threshold in thresholds:
```

```
    count = (
        dynamic_results["dynamic_score"] >= threshold
    ).sum()
```

```
    threshold_results.append({
        "threshold": threshold,
        "anomaly_count": count
    })
```

```
threshold_table = pd.DataFrame(threshold_results)
```

```
print(threshold_table)
```

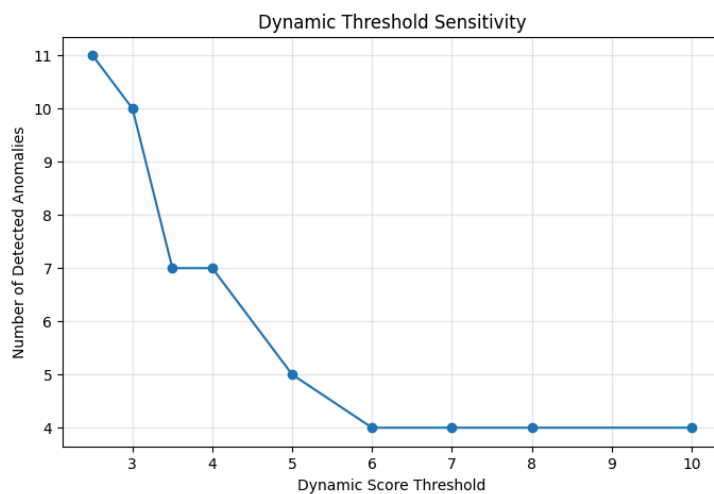
```
plt.figure(figsize=(8, 5))
```

```
plt.plot(
    threshold_table["threshold"],
    threshold_table["anomaly_count"],
    marker="o"
)
```

```
plt.xlabel("Dynamic Score Threshold")
plt.ylabel("Number of Detected Anomalies")
plt.title("Dynamic Threshold Sensitivity")
plt.grid(True, alpha=0.3)
```

```
plt.show()
```

	threshold	anomaly_count
0	2.5	11
1	3.0	10
2	3.5	7
3	4.0	7
4	5.0	5
5	6.0	4
6	7.0	4
7	8.0	4
8	10.0	4



```
median_score = dynamic_results["dynamic_score"].median()
```

```
mad_score = (
```

```

dynamic_results["dynamic_score"] - median_score
).abs().median()

dynamic_threshold = median_score + 3 * mad_score

print("Median:", round(median_score, 4))
print("MAD:", round(mad_score, 4))
print("Final dynamic threshold:", round(dynamic_threshold, 4))

dynamic_results["dynamic_flag"] = np.where(
    dynamic_results["dynamic_score"] >= dynamic_threshold,
    "ANOMALY",
    "NORMAL"
)

print(
    dynamic_results[
        ["curve_id", "dynamic_score", "dynamic_flag"]
    ].sort_values(
        "dynamic_score",
        ascending=False
    ).round(4)
)

```

```

Median: 3.1509
MAD: 1.644
Final dynamic threshold: 8.0831

```

	curve_id	dynamic_score	dynamic_flag
9	9	31.1230	ANOMALY
2	2	25.4660	ANOMALY
6	6	18.8380	ANOMALY
8	8	10.0008	ANOMALY
17	17	5.3960	NORMAL
7	7	4.8564	NORMAL
5	5	4.1412	NORMAL
0	0	3.4486	NORMAL
3	3	3.2376	NORMAL
10	10	3.1509	NORMAL
16	16	2.8316	NORMAL
15	15	2.3559	NORMAL
11	11	1.8935	NORMAL
1	1	1.6188	NORMAL
4	4	1.6188	NORMAL
14	14	1.5069	NORMAL
12	12	1.3978	NORMAL
18	18	1.3488	NORMAL
13	13	1.3475	NORMAL

```

from sklearn.ensemble import IsolationForest
from sklearn.preprocessing import StandardScaler

feature_columns = [
    col for col in breakdown_features.columns
    if col != "curve_id"
]

X = breakdown_features[feature_columns].copy()

# Standardize features
feature_scaler = StandardScaler()
X_scaled = feature_scaler.fit_transform(X)

contamination_values = [0.05, 0.10, 0.15, 0.20, 0.25]

contamination_results = []

for contamination in contamination_values:
    iso = IsolationForest(
        n_estimators=300,
        contamination=contamination,
        random_state=42
    )

    predictions = iso.fit_predict(X_scaled)

    anomaly_count = (predictions == -1).sum()

    contamination_results.append({
        "contamination": contamination,
        "anomaly_count": anomaly_count
    })

contamination_table = pd.DataFrame(contamination_results)

print(contamination_table)

```

	contamination	anomaly_count
0	0.05	1
1	0.10	2
2	0.15	3
3	0.20	4
4	0.25	5

```

plt.figure(figsize=(8, 5))

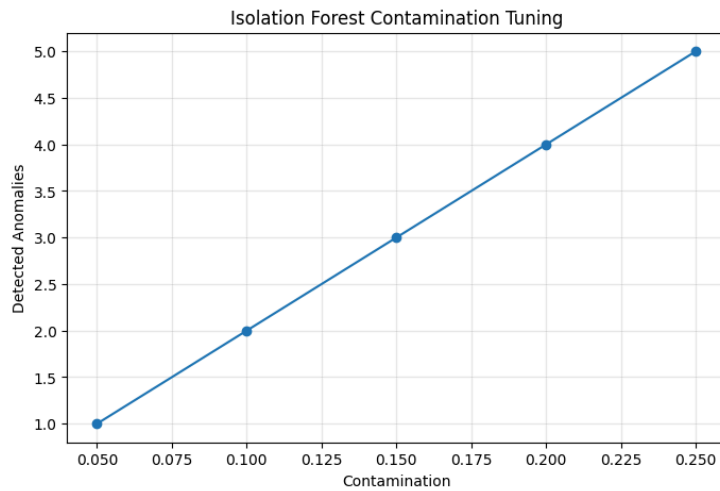
plt.plot(
    contamination_table["contamination"],
    contamination_table["anomaly_count"],
    marker="o"
)

plt.xlabel("Contamination")
plt.ylabel("Detected Anomalies")

```

```
plt.title("Isolation Forest Contamination Tuning")
plt.grid(True, alpha=0.3)

plt.show()
```



```
BEST_CONTAMINATION = 0.10

isolation_forest = IsolationForest(
    n_estimators=500,
    contamination=BEST_CONTAMINATION,
    random_state=42
)

isolation_forest.fit(X_scaled)

# Isolation Forest prediction
iso_prediction = isolation_forest.predict(X_scaled)

# Decision function:
# higher = more normal
# lower = more anomalous
iso_decision = isolation_forest.decision_function(X_scaled)

# Convert so higher = more anomalous
isolation_anomaly_score = -iso_decision

ml_results = pd.DataFrame({
    "curve_id": breakdown_features["curve_id"],
    "isolation_score": isolation_anomaly_score,
    "ml_flag": np.where(
        iso_prediction == -1,
        "ANOMALY",
        "NORMAL"
    )
})

print(
    ml_results.sort_values(
        "isolation_score",
        ascending=False
    ).round(4)
)
```

	curve_id	isolation_score	ml_flag
9	9	0.0890	ANOMALY
8	8	0.0738	ANOMALY
5	5	-0.0184	NORMAL
7	7	-0.0365	NORMAL
16	16	-0.0475	NORMAL
17	17	-0.0749	NORMAL
2	2	-0.0768	NORMAL
15	15	-0.0800	NORMAL
10	10	-0.0859	NORMAL
6	6	-0.0941	NORMAL
0	0	-0.0957	NORMAL
11	11	-0.1002	NORMAL
4	4	-0.1005	NORMAL
3	3	-0.1054	NORMAL
12	12	-0.1097	NORMAL
1	1	-0.1229	NORMAL
14	14	-0.1258	NORMAL
18	18	-0.1287	NORMAL
13	13	-0.1394	NORMAL

```
dynamic_scores = dynamic_results[
    ["curve_id", "dynamic_score"]
].copy()

# Normalize dynamic score relative to the selected threshold
dynamic_scores["dynamic_normalized"] = (
    dynamic_scores["dynamic_score"]
    / dynamic_threshold
)

dynamic_scores["dynamic_normalized"] = (
    dynamic_scores["dynamic_normalized"]
    .clip(0, 3)
)
```

```

iso_scores = ml_results[
    ["curve_id", "isolation_score"]
].copy()

iso_min = iso_scores["isolation_score"].min()
iso_max = iso_scores["isolation_score"].max()

iso_scores["isolation_normalized"] = (
    (iso_scores["isolation_score"] - iso_min)
    / (iso_max - iso_min + 1e-12)
)

```

```

combined_results = dynamic_scores.merge(
    iso_scores,
    on="curve_id"
)

combined_results["combined_score"] = (
    0.60 * combined_results["dynamic_normalized"]
    +
    0.40 * combined_results["isolation_normalized"]
)

combined_results = combined_results.sort_values(
    "combined_score",
    ascending=False
)

print(
    combined_results.round(4)
)

```

	curve_id	dynamic_score	dynamic_normalized	isolation_score \
9	9	31.1230	3.0000	0.0890
2	2	25.4660	3.0000	-0.0768
6	6	18.8380	2.3306	-0.0941
8	8	10.0008	1.2372	0.0738
7	7	4.8564	0.6008	-0.0365
5	5	4.1412	0.5123	-0.0184
17	17	5.3960	0.6676	-0.0749
16	16	2.8316	0.3503	-0.0475
0	0	3.4486	0.4266	-0.0957
10	10	3.1509	0.3898	-0.0859
3	3	3.2376	0.4005	-0.1054
15	15	2.3559	0.2915	-0.0800
11	11	1.8935	0.2343	-0.1002
4	4	1.6188	0.2003	-0.1005
12	12	1.3978	0.1729	-0.1097
1	1	1.6188	0.2003	-0.1229
14	14	1.5069	0.1864	-0.1258
18	18	1.3488	0.1669	-0.1287
13	13	1.3475	0.1667	-0.1394

	isolation_normalized	combined_score
9	1.0000	2.2000
2	0.2742	1.9097
6	0.1986	1.4778
8	0.9332	1.1156
7	0.4504	0.5407
5	0.5295	0.5192
17	0.2826	0.5136
16	0.4022	0.3711
0	0.1916	0.3326
10	0.2343	0.3276
3	0.1491	0.3000
15	0.2600	0.2789
11	0.1717	0.2092
4	0.1705	0.1884
12	0.1300	0.1558
1	0.0725	0.1491
14	0.0596	0.1357
18	0.0470	0.1189
13	0.0000	0.1000

```

combined_results["final_decision"] = np.select(
    [
        combined_results["combined_score"] >= 1.00,
        combined_results["combined_score"] >= 0.70
    ],
    [
        "ANOMALY",
        "HOLD"
    ],
    default="PASS"
)

print(
    combined_results[
        [
            "curve_id",
            "dynamic_score",
            "dynamic_normalized",
            "isolation_score",
            "isolation_normalized",
            "combined_score",
            "final_decision"
        ]
    ].round(4)
)

```

	curve_id	dynamic_score	dynamic_normalized	isolation_score \
9	9	31.1230	3.0000	0.0890
2	2	25.4660	3.0000	-0.0768
6	6	18.8380	2.3306	-0.0941
8	8	10.0008	1.2372	0.0738
7	7	4.8564	0.6008	-0.0365

5	5	4.1412	0.5123	-0.0184
17	17	5.3960	0.6676	-0.0749
16	16	2.8316	0.3503	-0.0475
0	0	3.4486	0.4266	-0.0957
10	10	3.1509	0.3898	-0.0859
3	3	3.2376	0.4005	-0.1054
15	15	2.3559	0.2915	-0.0800
11	11	1.8935	0.2343	-0.1002
4	4	1.6188	0.2003	-0.1005
12	12	1.3978	0.1729	-0.1097
1	1	1.6188	0.2003	-0.1229
14	14	1.5069	0.1864	-0.1258
18	18	1.3488	0.1669	-0.1287
13	13	1.3475	0.1667	-0.1394

	isolation_normalized	combined_score	final_decision
9	1.0000	2.2000	ANOMALY
2	0.2742	1.9097	ANOMALY
6	0.1986	1.4778	ANOMALY
8	0.9332	1.1156	ANOMALY
7	0.4504	0.5407	PASS
5	0.5295	0.5192	PASS
17	0.2826	0.5136	PASS
16	0.4022	0.3711	PASS
0	0.1916	0.3326	PASS
10	0.2343	0.3276	PASS
3	0.1491	0.3000	PASS
15	0.2600	0.2789	PASS
11	0.1717	0.2092	PASS
4	0.1705	0.1884	PASS
12	0.1300	0.1558	PASS
1	0.0725	0.1491	PASS
14	0.0596	0.1357	PASS
18	0.0470	0.1189	PASS
13	0.0000	0.1000	PASS

```

final_anomaly_ids = combined_results[
    combined_results["final_decision"] == "ANOMALY"
]["curve_id"].tolist()

print("Final anomaly curves:", final_anomaly_ids)

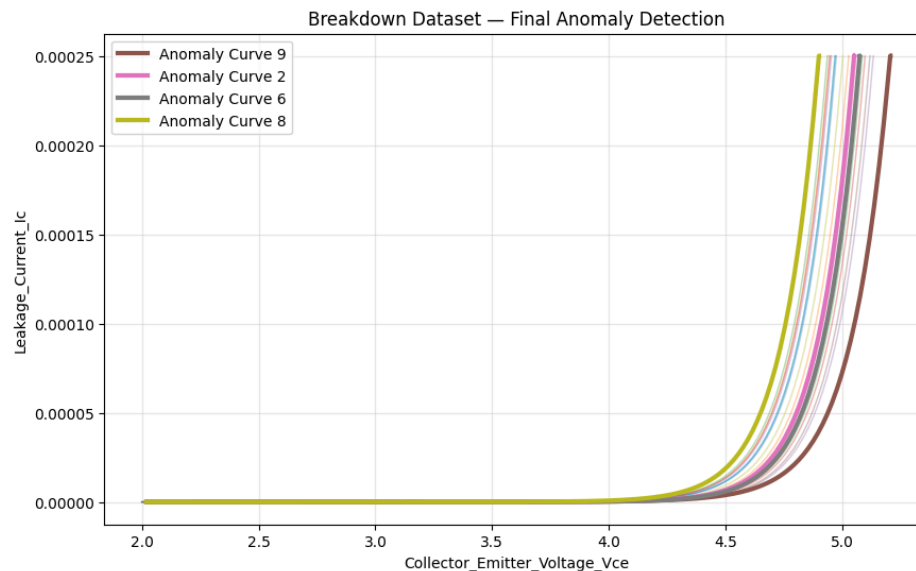
plot_anomalies(
    breakdown_curves,
    "Collector_Emitter_Voltage_Vce",
    "Leakage_Current_Ic",
    final_anomaly_ids,
    "Breakdown Dataset - Final Anomaly Detection"
)

hold_ids = combined_results[
    combined_results["final_decision"] == "HOLD"
]["curve_id"].tolist()

print("HOLD curves:", hold_ids)

```

Final anomaly curves: [9, 2, 6, 8]



HOLD curves: []

▼ ONE reusable pipeline function

```

def build_anomaly_model(
    curves,
    features,
    model_name,
    x_col,
    y_col,
    contamination=0.10
):

    print("=" * 70)
    print(f"{{model_name.upper()}} ANOMALY MODEL")
    print("=" * 70)

```



```

# -----
# 1. Feature preparation
# -----

feature_columns = [
    col for col in features.columns
    if col != "curve_id"
]

X = features[feature_columns].copy()

# -----
# 2. Standardization
# -----

scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)

# -----
# 3. Robust dynamic score
# -----

median_values = X.median()

mad_values = (
    X - median_values
).abs().median()

# Prevent division by zero
mad_values = mad_values.replace(0, 1e-12)

robust_z = (
    X - median_values
) / (
    1.4826 * mad_values
)

dynamic_score = robust_z.abs().max(axis=1)

dynamic_results = pd.DataFrame({
    "curve_id": features["curve_id"],
    "dynamic_score": dynamic_score
})

# -----
# 4. Dynamic threshold
# -----

score_median = dynamic_score.median()

score_mad = (
    dynamic_score - score_median
).abs().median()

dynamic_threshold = (
    score_median + 3 * score_mad
)

dynamic_results["dynamic_flag"] = np.where(
    dynamic_score >= dynamic_threshold,
    "ANOMALY",
    "NORMAL"
)

# -----
# 5. Isolation Forest
# -----

isolation_forest = IsolationForest(
    n_estimators=500,
    contamination=contamination,
    random_state=42
)

isolation_forest.fit(X_scaled)

predictions = isolation_forest.predict(X_scaled)

decision_values = (
    isolation_forest.decision_function(X_scaled)
)

isolation_score = -decision_values

ml_results = pd.DataFrame({
    "curve_id": features["curve_id"],
    "isolation_score": isolation_score,
    "ml_flag": np.where(
        predictions == -1,
        "ANOMALY",
        "NORMAL"
    )
})

# -----
# 6. Normalize scores
# -----

dynamic_normalized = (
    dynamic_score / dynamic_threshold
).clip(0, 3)

iso_min = isolation_score.min()

```

```

iso_max = isolation_score.max()

isolation_normalized = (
    (isolation_score - iso_min)
    / (iso_max - iso_min + 1e-12)
)

# -----
# 7. Combined score
# -----

combined_score = (
    0.60 * dynamic_normalized
    +
    0.40 * isolation_normalized
)

# -----
# 8. Final decision
# -----

final_decision = np.select(
    [
        combined_score >= 1.00,
        combined_score >= 0.70
    ],
    [
        "ANOMALY",
        "HOLD"
    ],
    default="PASS"
)

results = pd.DataFrame({
    "curve_id": features["curve_id"],
    "dynamic_score": dynamic_score,
    "dynamic_flag": dynamic_results["dynamic_flag"],
    "isolation_score": isolation_score,
    "ml_flag": ml_results["ml_flag"],
    "combined_score": combined_score,
    "final_decision": final_decision
})

results = results.sort_values(
    "combined_score",
    ascending=False
)

# -----
# 9. Print summary
# -----

print("\nDynamic threshold:",
      round(dynamic_threshold, 4))

print(
    "\nDynamic anomalies:",
    dynamic_results[
        dynamic_results["dynamic_flag"] == "ANOMALY"
    ]["curve_id"].tolist()
)

print(
    "Isolation Forest anomalies:",
    ml_results[
        ml_results["ml_flag"] == "ANOMALY"
    ]["curve_id"].tolist()
)

print(
    "Final anomalies:",
    results[
        results["final_decision"] == "ANOMALY"
    ]["curve_id"].tolist()
)

print("\nFinal results:")
print(results.round(4))

# -----
# 10. Visualization
# -----

anomaly_ids = results[
    results["final_decision"] == "ANOMALY"
]["curve_id"].tolist()

plot_anomalies(
    curves,
    x_col,
    y_col,
    anomaly_ids,
    f"{model_name} - Final Anomaly Detection"
)

return {
    "scaler": scaler,
    "isolation_forest": isolation_forest,
    "results": results,
    "dynamic_threshold": dynamic_threshold,
    "feature_columns": feature_columns
}

```

```
#For breakdown

breakdown_model = build_anomaly_model(
    curves=breakdown_curves,
    features=breakdown_features,
    model_name="Breakdown",
    x_col="Collector_Emitter_Voltage_Vce",
    y_col="Leakage_Current_Ic",
    contamination=0.10
)
```

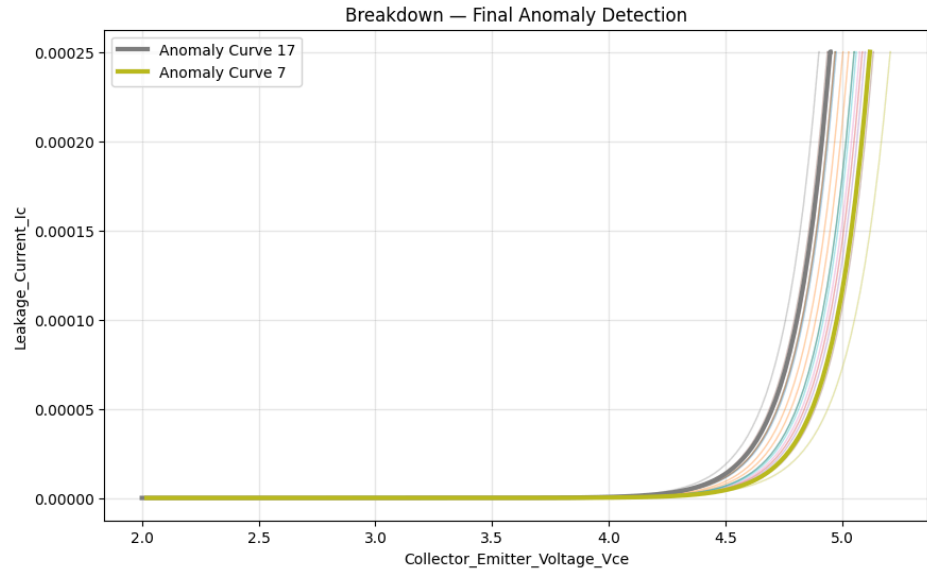
```
=====
BREAKDOWN ANOMALY MODEL
=====

Dynamic threshold: 353.5505

Dynamic anomalies: [3, 7, 17]
Isolation Forest anomalies: [8, 9]
Final anomalies: [17, 7]

Final results:
curve_id dynamic_score dynamic_flag isolation_score ml_flag \
17 17 674.4908 ANOMALY -0.0749 NORMAL
7 7 607.0417 ANOMALY -0.0365 NORMAL
8 8 269.7963 NORMAL 0.0738 ANOMALY
3 3 404.6945 ANOMALY -0.1054 NORMAL
5 5 269.7963 NORMAL -0.0184 NORMAL
16 16 290.7055 NORMAL -0.0475 NORMAL
9 9 134.8982 NORMAL 0.0890 ANOMALY
15 15 290.9255 NORMAL -0.0800 NORMAL
10 10 290.9841 NORMAL -0.0859 NORMAL
11 11 291.4387 NORMAL -0.1002 NORMAL
12 12 290.7348 NORMAL -0.1097 NORMAL
14 14 291.0428 NORMAL -0.1258 NORMAL
18 18 291.3507 NORMAL -0.1287 NORMAL
13 13 291.0721 NORMAL -0.1394 NORMAL
4 4 202.3472 NORMAL -0.1005 NORMAL
1 1 202.3472 NORMAL -0.1229 NORMAL
2 2 134.8982 NORMAL -0.0768 NORMAL
6 6 22.3068 NORMAL -0.0941 NORMAL
0 0 3.7116 NORMAL -0.0957 NORMAL

combined_score final_decision
17 1.2577 ANOMALY
7 1.2104 ANOMALY
8 0.8311 HOLD
3 0.7465 HOLD
5 0.6697 PASS
16 0.6542 PASS
9 0.6289 PASS
15 0.5977 PASS
10 0.5875 PASS
11 0.5633 PASS
12 0.5454 PASS
14 0.5178 PASS
18 0.5133 PASS
13 0.4940 PASS
4 0.4116 PASS
1 0.3724 PASS
2 0.3386 PASS
6 0.1173 PASS
0 0.0829 PASS
```



```
##Leakage IV

leakage_model = build_anomaly_model(
    curves=leakage_curves,
    features=leakage_features,
    model_name="Leakage IV",
    x_col="Applied_Voltage",
    y_col="Leakage_Current",
    contamination=0.10
)
```

LEAKAGE IV ANOMALY MODEL

Dynamic threshold: 4291.6298

Dynamic anomalies: [2]

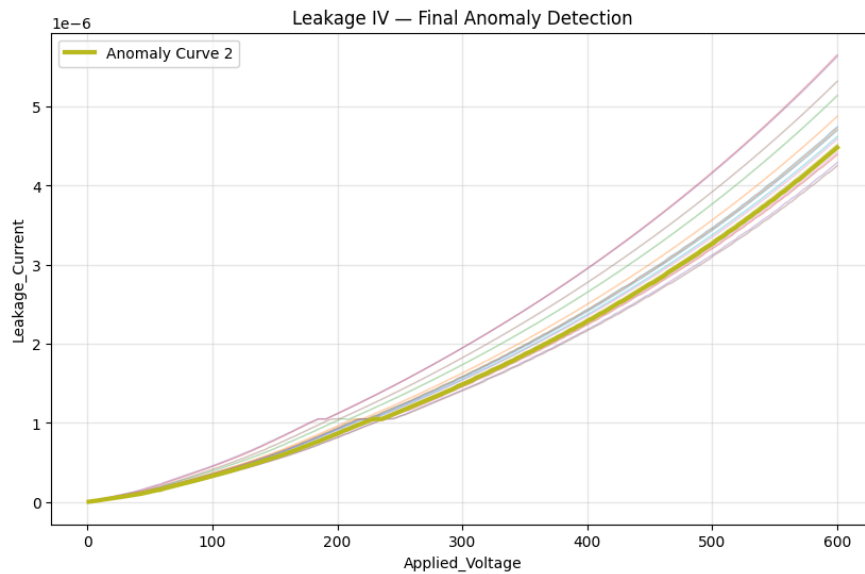
Isolation Forest anomalies: [14, 15]

Final anomalies: [2]

Final results:

	curve_id	dynamic_score	dynamic_flag	isolation_score	ml_flag	\
2	2	23575.4102	ANOMALY	-0.1282	NORMAL	
15	15	3571.3095	NORMAL	0.0232	ANOMALY	
14	14	2378.2147	NORMAL	0.0348	ANOMALY	
16	16	2378.7305	NORMAL	-0.0058	NORMAL	
7	7	3926.7764	NORMAL	-0.1473	NORMAL	
3	3	3926.7982	NORMAL	-0.1655	NORMAL	
5	5	1967.5548	NORMAL	-0.0725	NORMAL	
6	6	1967.4678	NORMAL	-0.0748	NORMAL	
13	13	1191.9839	NORMAL	-0.0434	NORMAL	
1	1	1193.3329	NORMAL	-0.0731	NORMAL	
18	18	1192.0632	NORMAL	-0.0784	NORMAL	
0	0	1317.0629	NORMAL	-0.0876	NORMAL	
4	4	1967.2067	NORMAL	-0.1581	NORMAL	
8	8	1967.1632	NORMAL	-0.1703	NORMAL	
17	17	1192.3806	NORMAL	-0.1173	NORMAL	
10	10	1192.3410	NORMAL	-0.1534	NORMAL	
12	12	1.3900	NORMAL	-0.0864	NORMAL	
11	11	1.4680	NORMAL	-0.1319	NORMAL	
9	9	0.9274	NORMAL	-0.1451	NORMAL	

	combined_score	final_decision
2	1.8820	ANOMALY
15	0.8766	HOLD
14	0.7325	HOLD
16	0.6533	PASS
7	0.5939	PASS
3	0.5583	PASS
5	0.4658	PASS
6	0.4612	PASS
13	0.4141	PASS
1	0.3563	PASS
18	0.3459	PASS
0	0.3454	PASS
4	0.2988	PASS
8	0.2750	PASS
17	0.2700	PASS
10	0.1996	PASS
12	0.1638	PASS
11	0.0750	PASS
9	0.0493	PASS



```
##Turn on
turnon_model = build_anomaly_model(
    curves=turnon_curves,
    features=turnon_features,
    model_name="Turn-On",
    x_col="Gate_Voltage",
    y_col="Collector_Current",
    contamination=0.10
)
```